RL support in vLLM

Kaichao You @ vLLM Team

个人简介

本科

清华大学软件学院
软件工程专业，工学学士

访学经历

加州大学伯克利分校访问学生，
指导老师 Michael I. Jordan

开源经历

antlr4, tianshou等开源项目



研究生

清华大学软件学院
软件工程专业，工学博士

访学经历

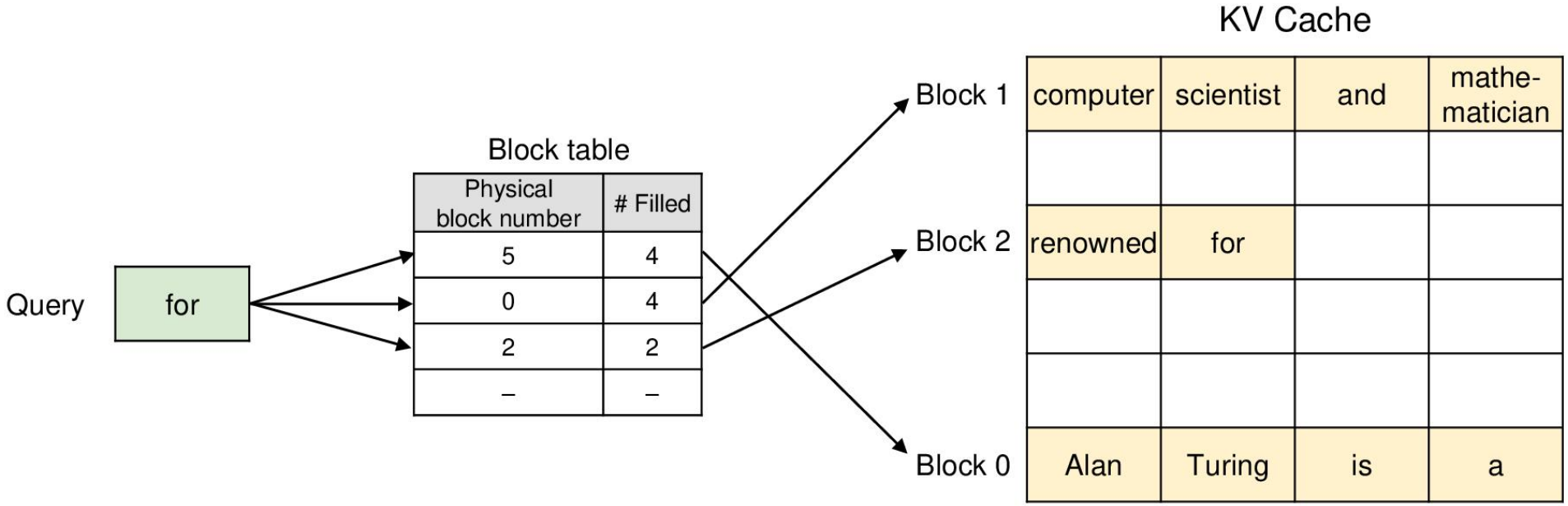
加州大学伯克利分校访问学生，
指导老师 Ion Stoica

开源经历

PyTorch、vLLM等开源项目

vLLM: From PagedAttention

An attention algorithm that allows for storing continuous keys and values in non-contiguous

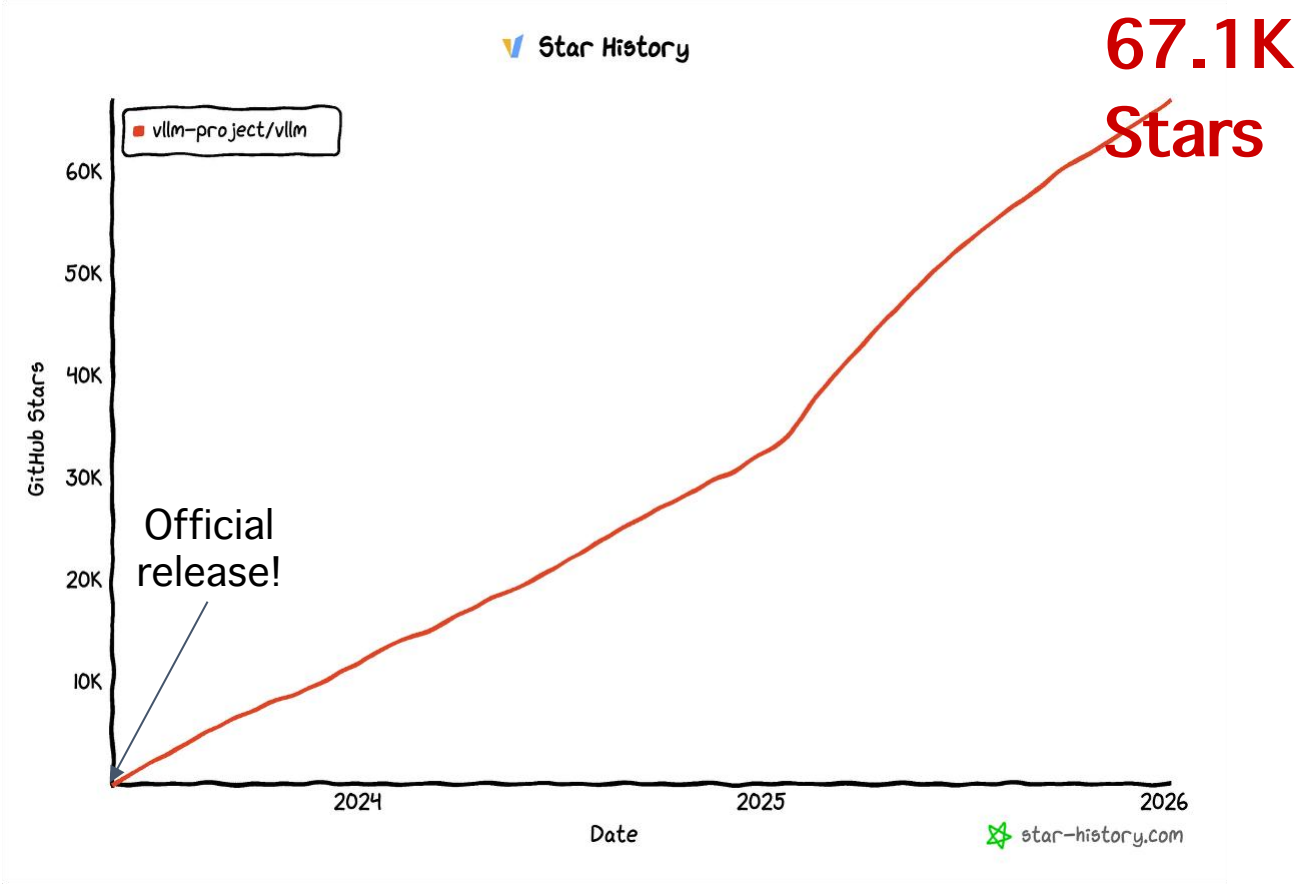


Woosuk Kwon, etc. Efficient Memory Management for Large Language Model Serving with PagedAttention, SOSP 2023

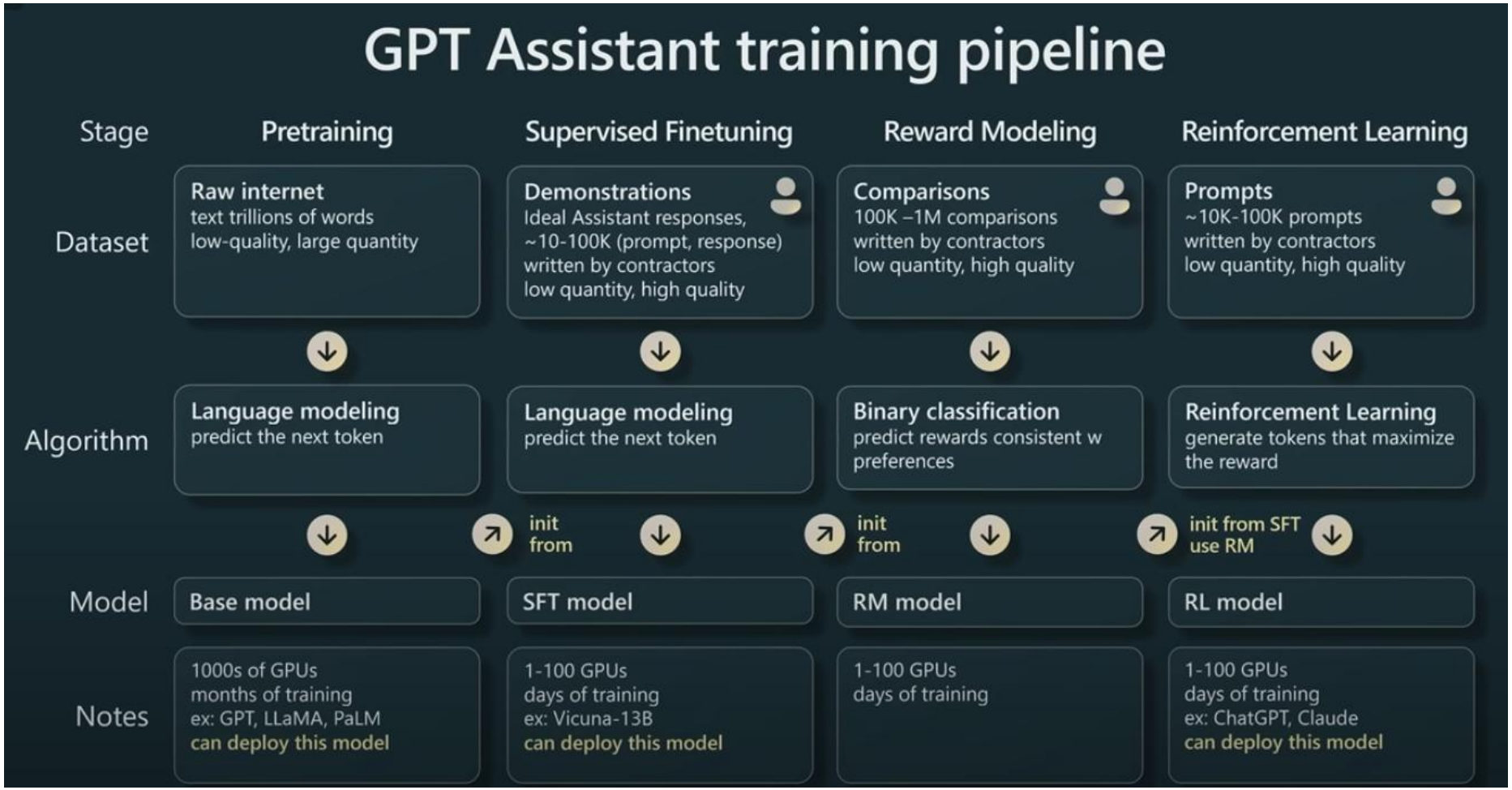


$ pip install vllm

vLLM

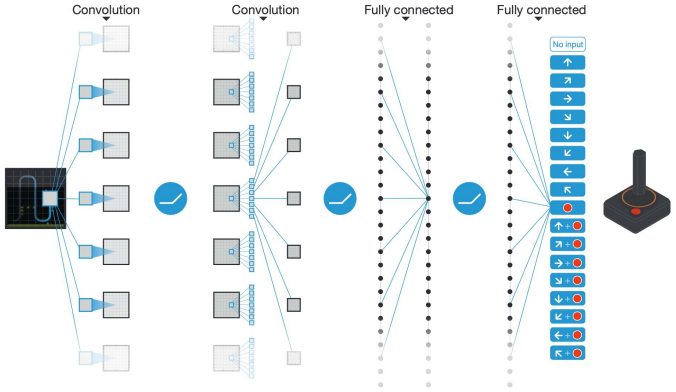


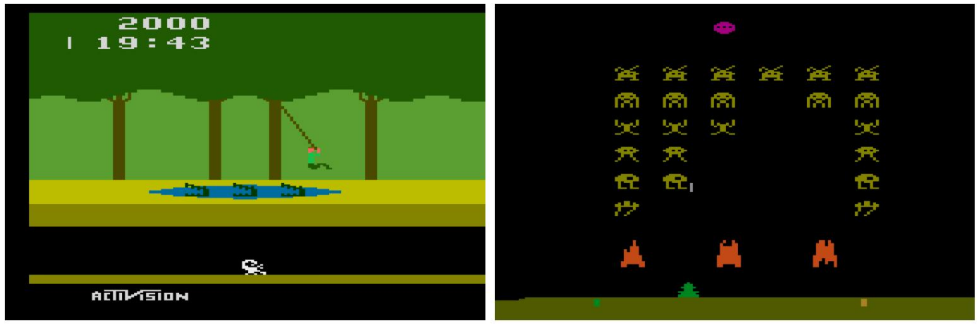
Connection Between RL and vLLM

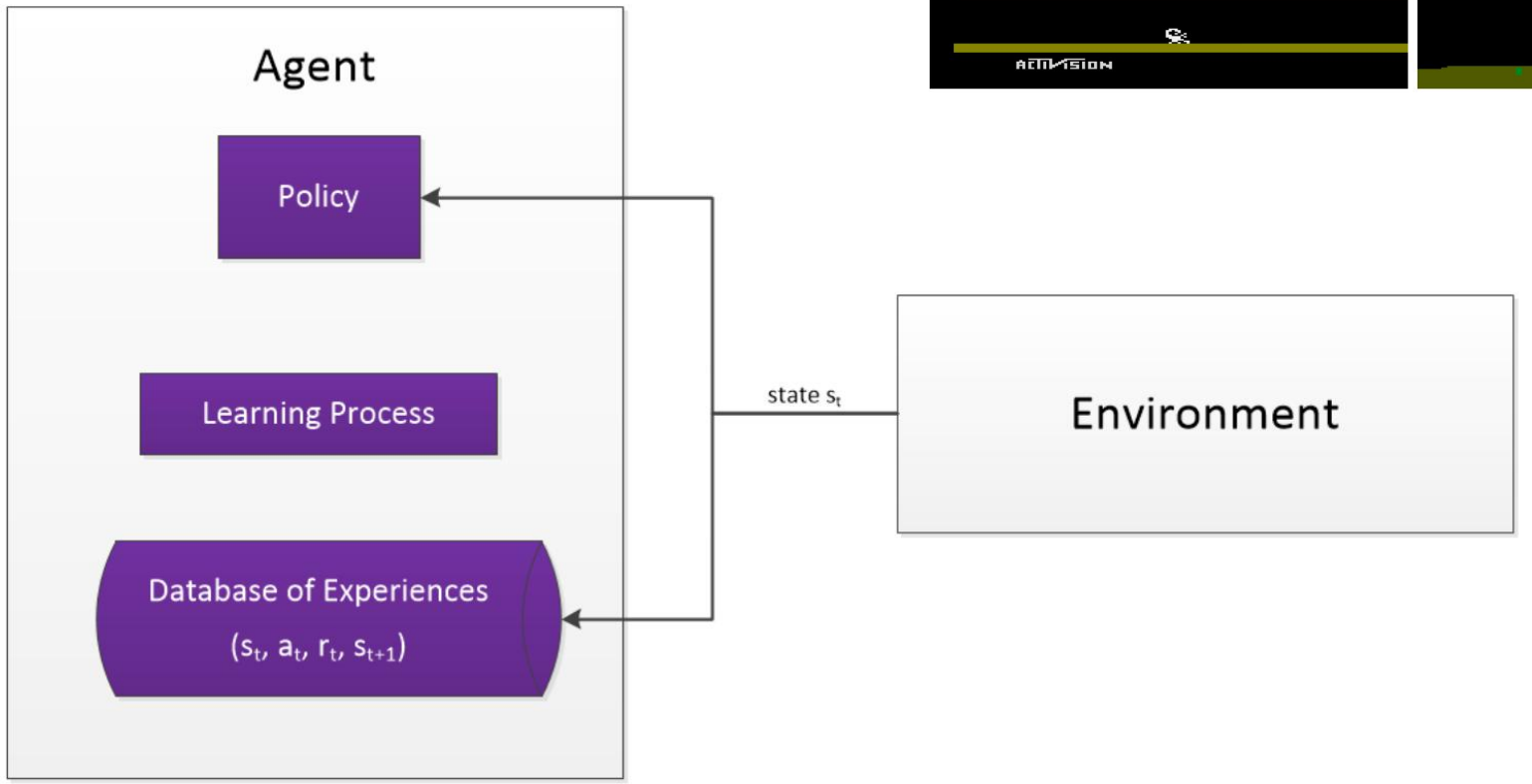


Andrej Karpathy. State of GPT, Microsoft Build 2023

RL in 2010-2020

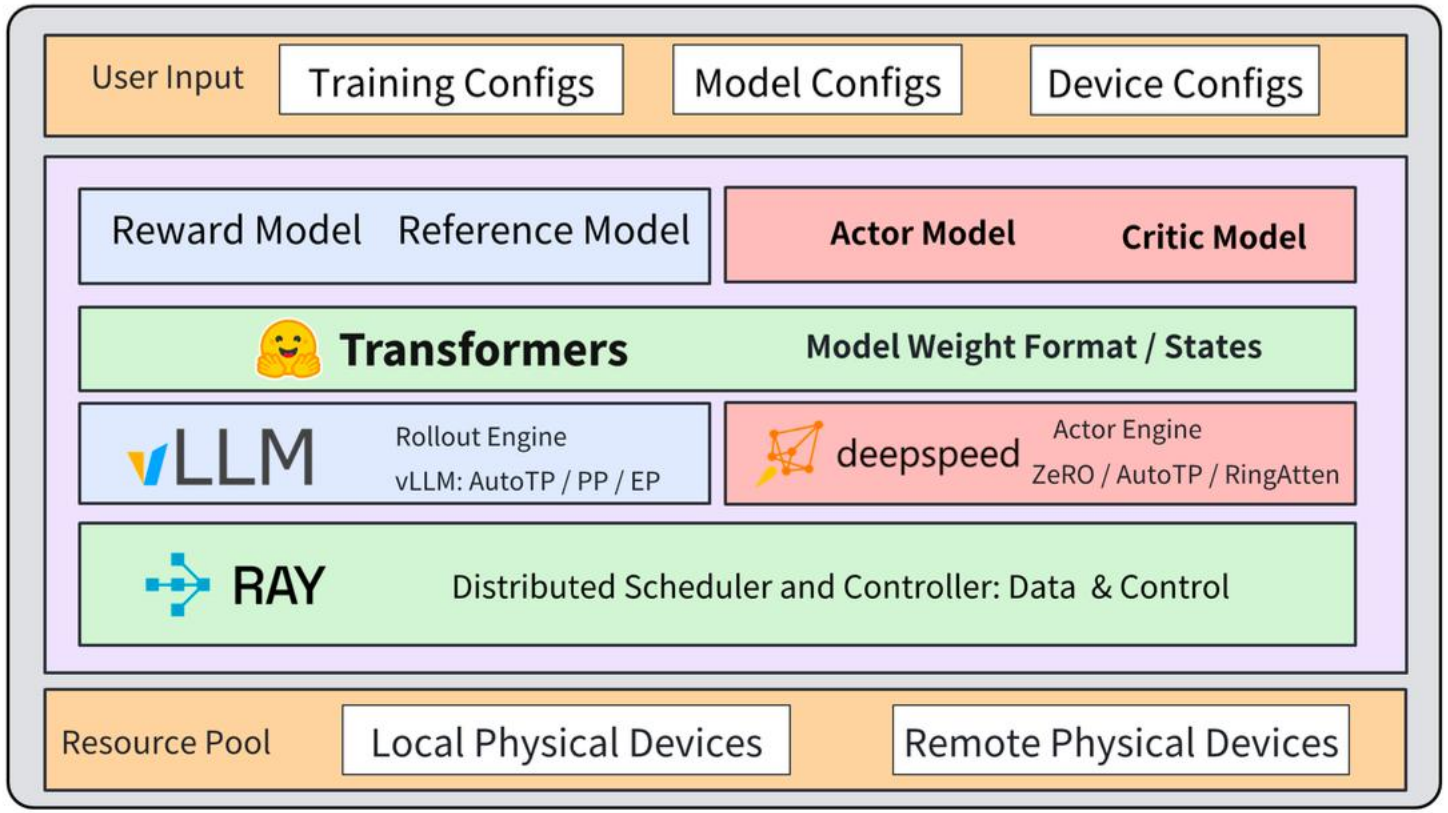






https://tianshou.org/en/stable/01_user_guide/00_training_process.html

RLHF Milestone: OpenRLHF

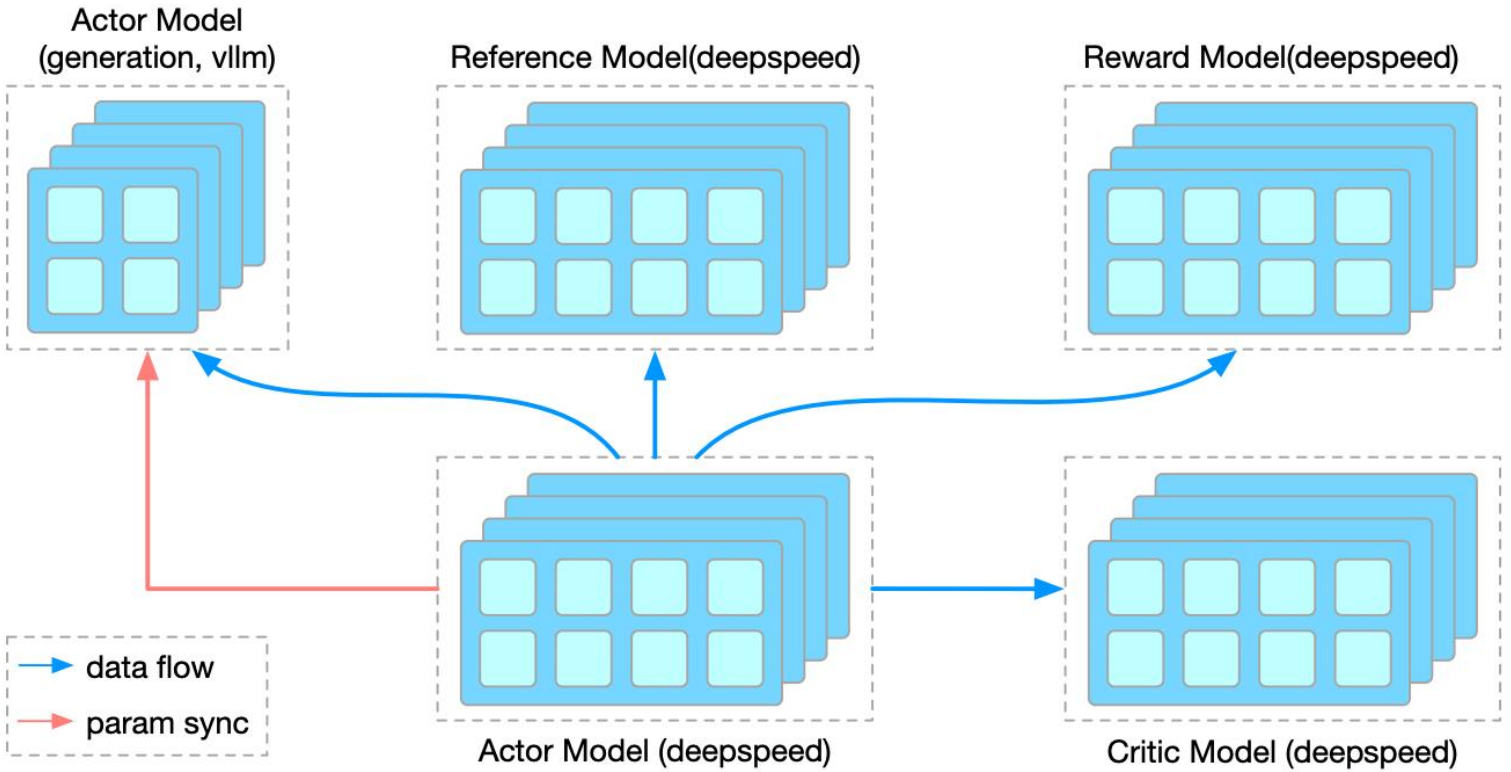


Size	GPUs	Generation	vLLM Weights Sync	Get Logits, Reward	Training	Total
7B	16	262.96	4.32	32.7	171.13	471.11
13B	32	372.14	10.03	29.58	197.18	608.93
34B	32	720.50	35.47	326.00	444.43	1526.40
70B	32	2252.79	111.65	323.38	1800.71	4488.53

Table 5: The time consumption (seconds) of each stage of PPO to train 1024 prompts with 1 PPO epoch in OpenRLHF.

Jian Hu, etc. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework

Disjoint Resource Allocation

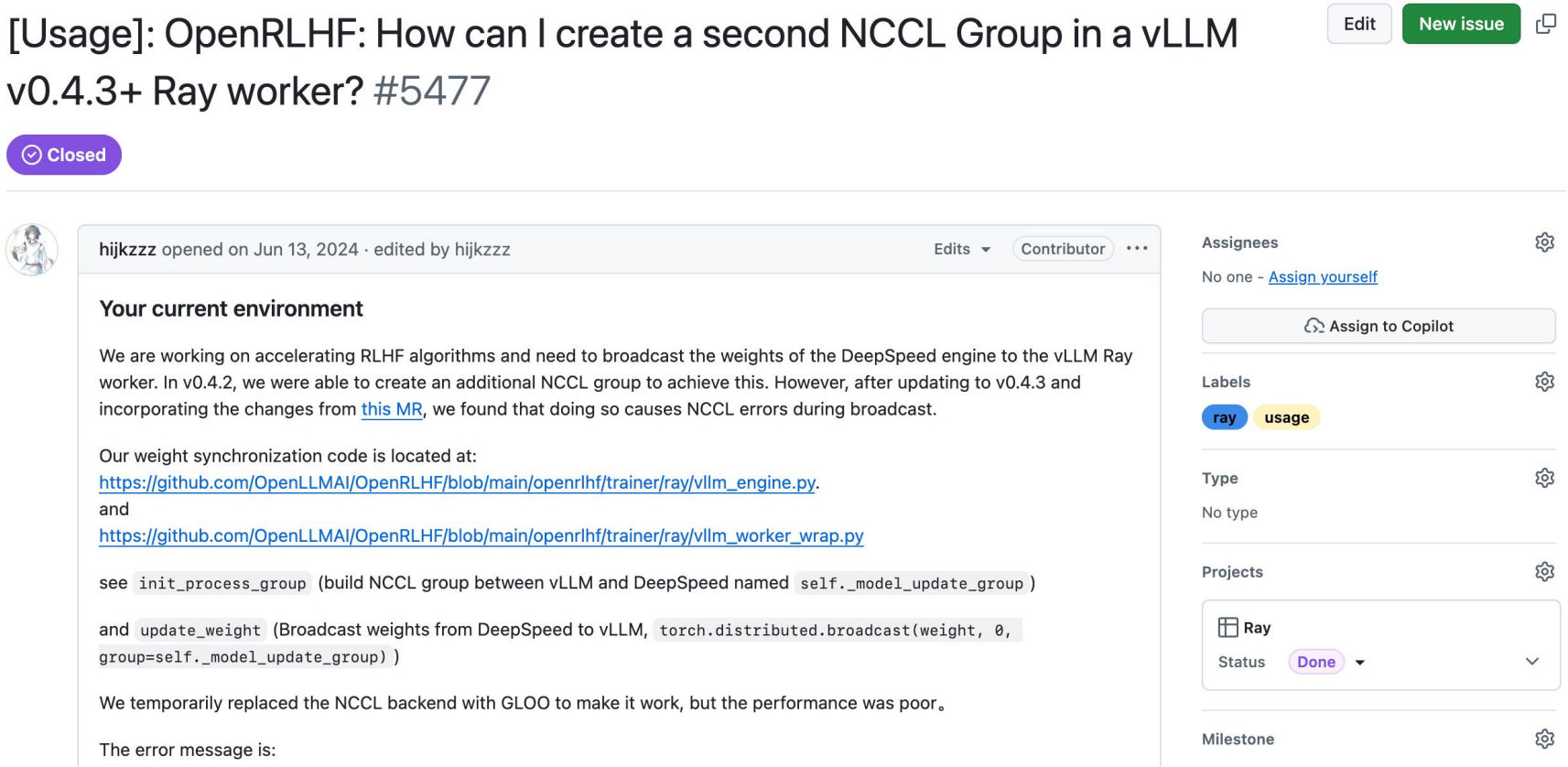


vllm > model_executor > models > llama.py > LlamaModel > load_weights

```
371    class LlamaModel(nn.Module):
455        def load_weights(self, weights: Iterable[tuple[str, torch.Tensor]])
```

Jian Hu, etc. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework

First Notification From OpenRLHF in vLLM



[Usage]: OpenRLHF: How can I create a second NCCL Group in a vLLM v0.4.3+ Ray worker? #5477

Edit | New issue

Closed

hijkzzz opened on Jun 13, 2024 · edited by hijkzzz — Edits — Contributor

Your current environment

We are working on accelerating RLHF algorithms and need to broadcast the weights of the DeepSpeed engine to the vLLM Ray worker. In v0.4.2, we were able to create an additional NCCL group to achieve this. However, after updating to v0.4.3 and incorporating the changes from this MR, we found that doing so causes NCCL errors during broadcast.

Our weight synchronization code is located at:
https://github.com/OpenLLMAI/OpenRLHF/blob/main/openrlhf/trainer/ray/vllm_engine.py.
and
https://github.com/OpenLLMAI/OpenRLHF/blob/main/openrlhf/trainer/ray/vllm_worker_wrap.py

see `init_process_group` (build NCCL group between vLLM and DeepSpeed named `self._model_update_group`)

and `update_weight` (Broadcast weights from DeepSpeed to vLLM, `torch.distributed.broadcast(weight, 0, group=self._model_update_group)`)

We temporarily replaced the NCCL backend with GLOO to make it work, but the performance was poor。

The error message is:

Assignees: No one - Assign yourself

Assign to Copilot

Labels: ray, usage

Type: No type

Projects: Ray — Status: Done

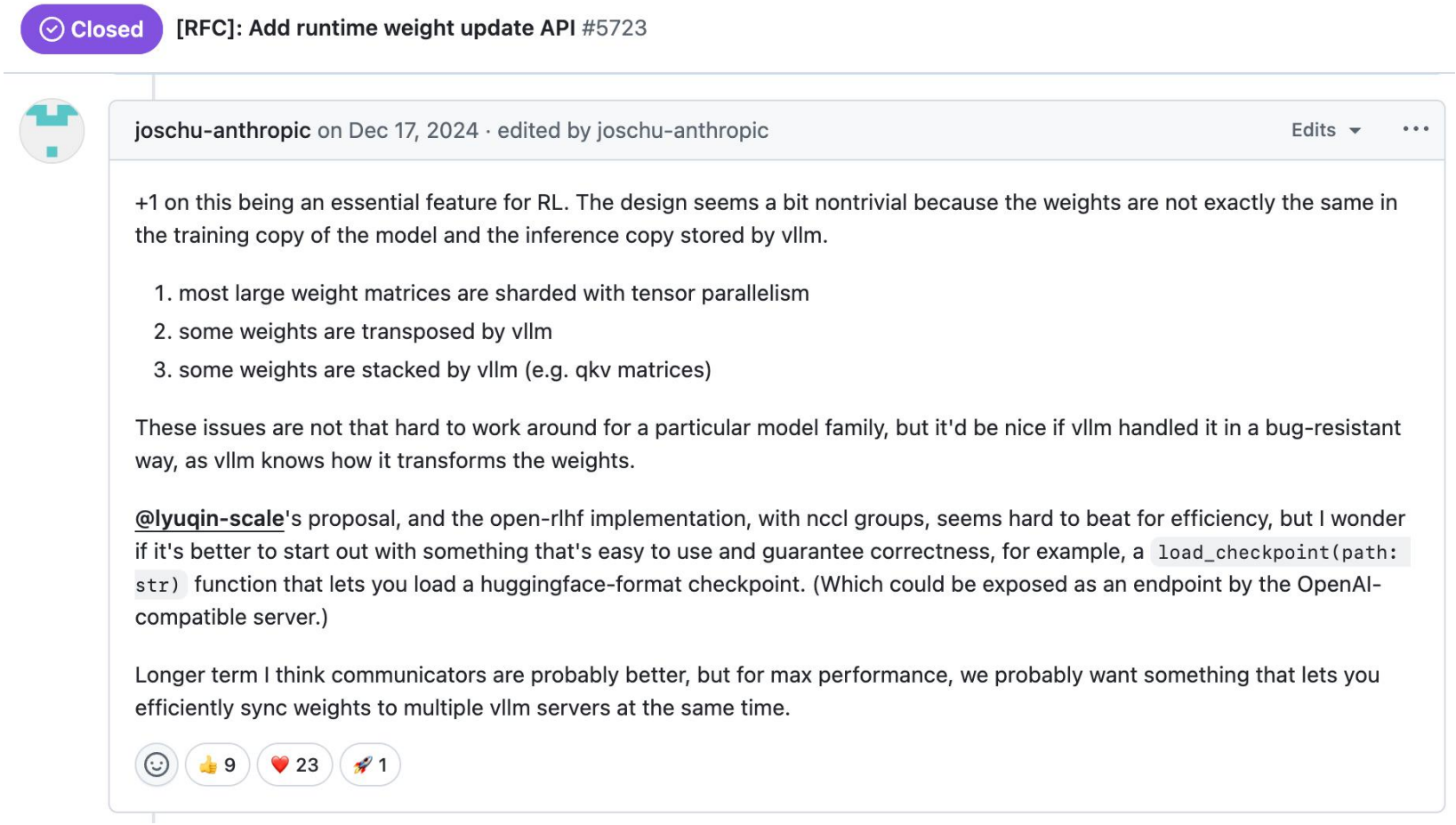
Milestone

[RFC]: Add runtime weight update API #5723

Closed

lyuqin-scale opened on Jun 21, 2024

Escalated Priority By John Schulman



Closed **[RFC]: Add runtime weight update API** #5723

joschu-anthropic on Dec 17, 2024 · edited by joschu-anthropic — Edits

+1 on this being an essential feature for RL. The design seems a bit nontrivial because the weights are not exactly the same in the training copy of the model and the inference copy stored by vllm.

1. most large weight matrices are sharded with tensor parallelism
2. some weights are transposed by vllm
3. some weights are stacked by vllm (e.g. qkv matrices)

These issues are not that hard to work around for a particular model family, but it'd be nice if vllm handled it in a bug-resistant way, as vllm knows how it transforms the weights.

@lyuqin-scale's proposal, and the open-rlhf implementation, with nccl groups, seems hard to beat for efficiency, but I wonder if it's better to start out with something that's easy to use and guarantee correctness, for example, a `load_checkpoint(path: str)` function that lets you load a huggingface-format checkpoint. (Which could be exposed as an endpoint by the OpenAI-compatible server.)

Longer term I think communicators are probably better, but for max performance, we probably want something that lets you efficiently sync weights to multiple vllm servers at the same time.

👍 9 ❤️ 23 🚀 1

Weight Transfer Solved After A Minimal Repro

Closed **[RFC]: Add runtime weight update API** #5723



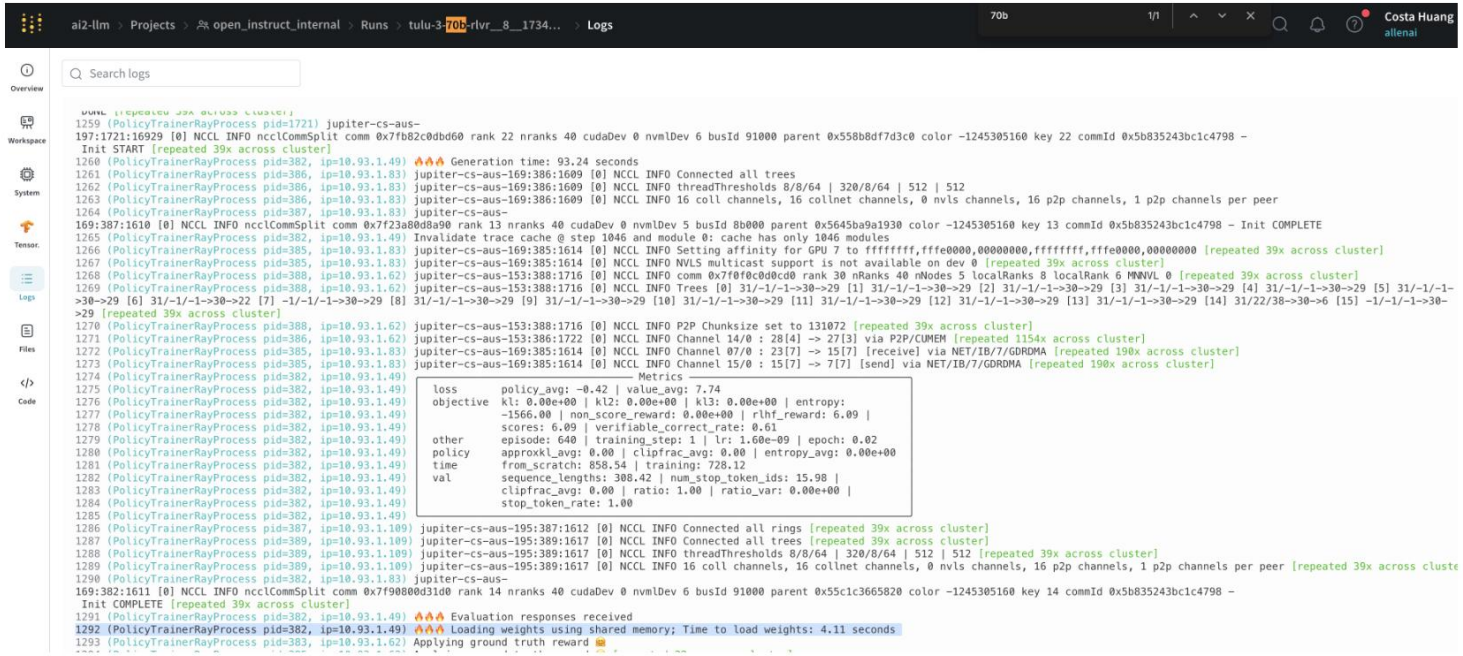
vwxyzjn on Dec 19, 2024 · edited by vwxyzjn Edits

Hi **@zhuzilin**, **@youkaichao** and I were looking at the NCCL issues together yesterday. Basically, the NCCL backend hangs or errors out with `Cuda failure 'invalid argument'` (see here for the minimal repro)

@youkaichao then provided an awesome fix 🔥: just set `export NCCL_CUMEM_ENABLE=0`

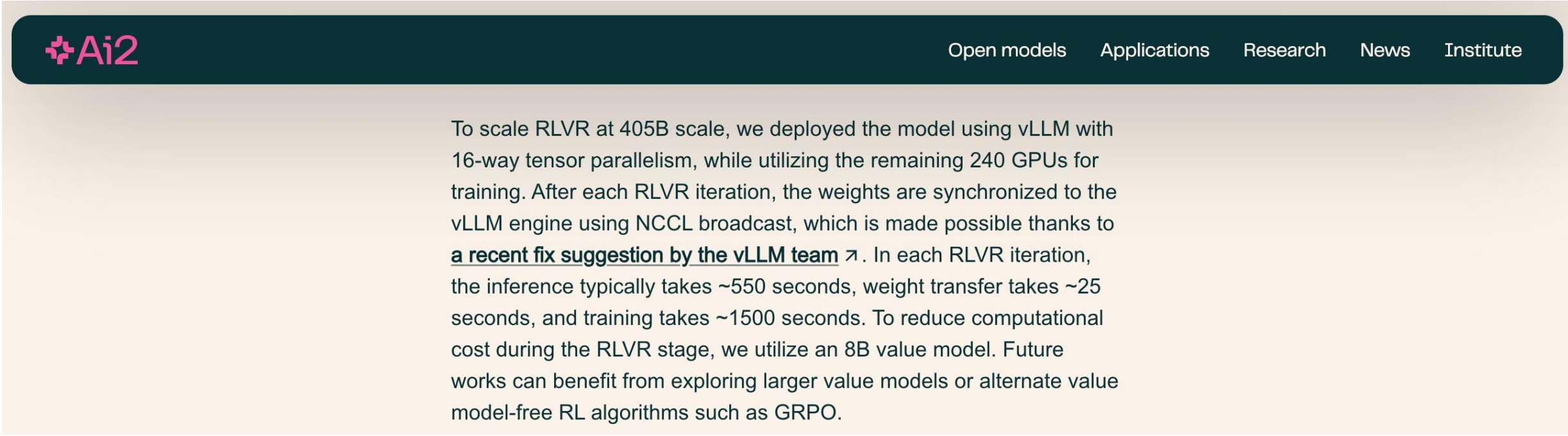
We now see substantial improvement for transferring weights, especially in 70B (45x faster)

- 70B: time to load weights go from 180 seconds using `gloo` to 4 seconds using `nccl`
- 8B: time to load weights go from 5-6 seconds using `gloo` to 0.85 seconds using `nccl`



End-to-end wise it's already 40% faster (34089 / 24620 episodes)

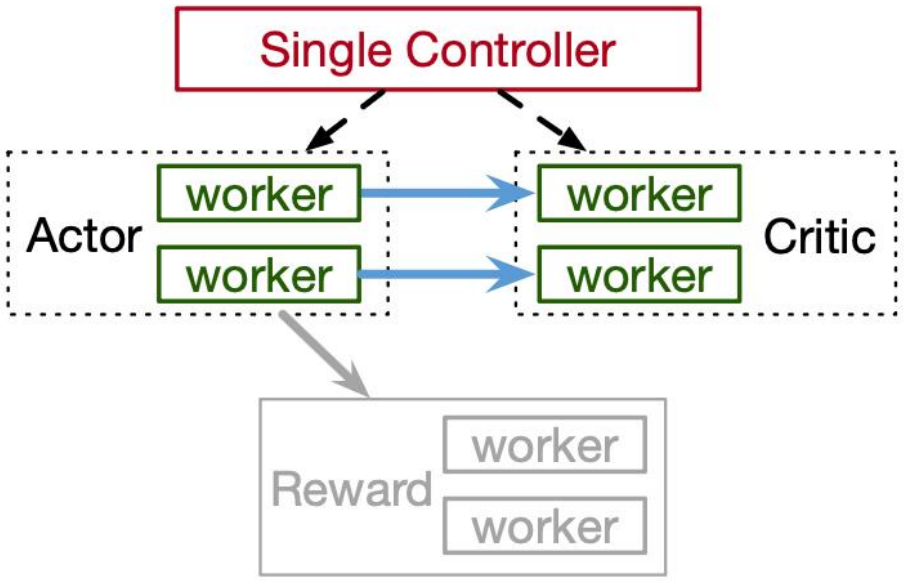
Result: Successful 405B RL

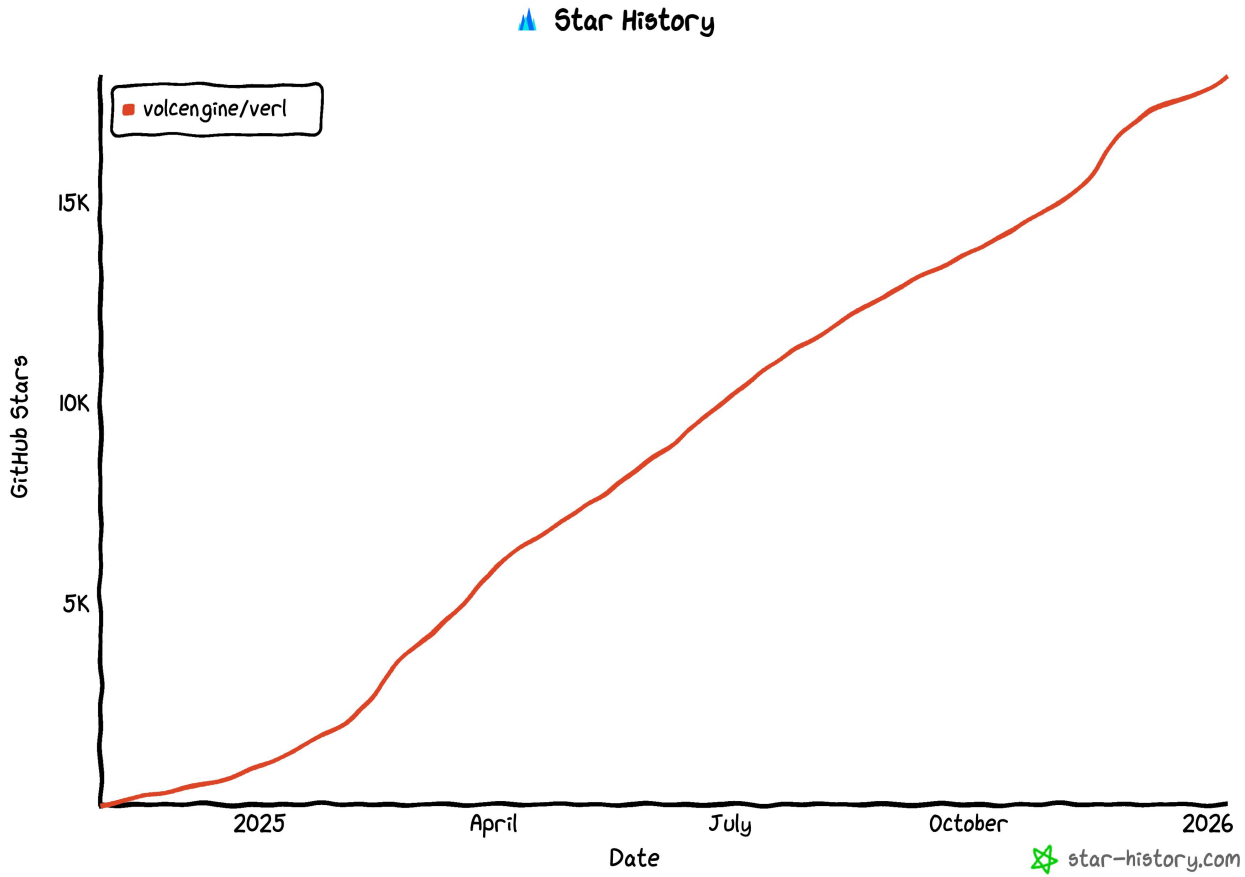


https://allenai.org/blog/tulu-3-405b



Rise of Verl


Single Controller
Actor
worker
worker
worker
worker
Critic
Reward
worker
worker


Star History
volcengine/verl
15K
10K
5K
GitHub Stars
2025
April
July
October
2026
Date
star-history.com

vLLM changes for Verl

[RFC]: Fully SPMD Execution for Offline Inference #11400

Closed

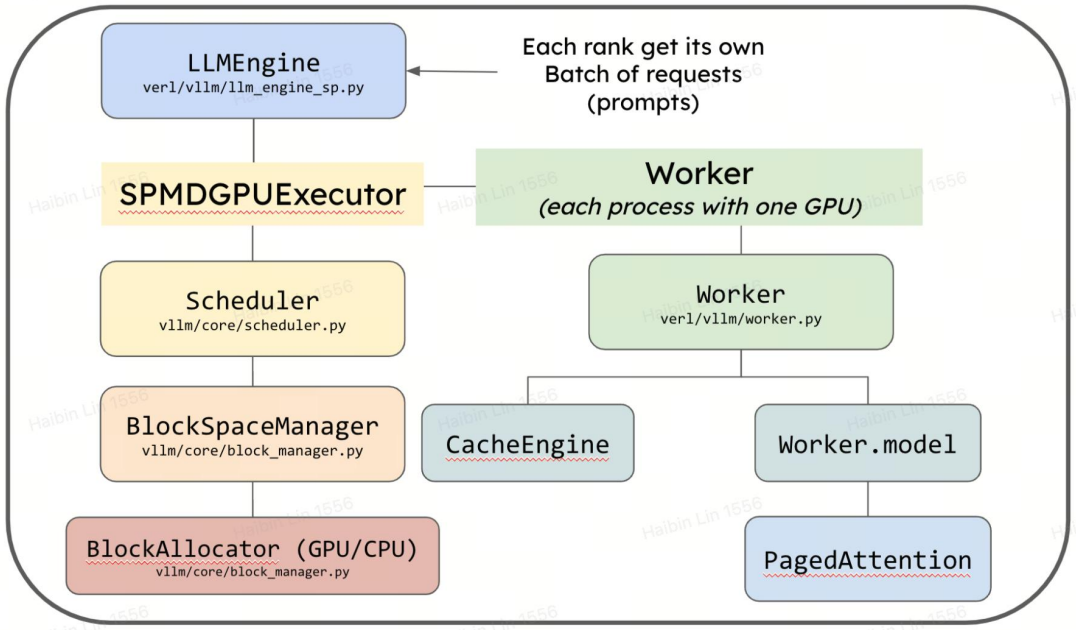
eric-haibin-lin opened on Dec 22, 2024 · edited by eric-haibin-lin Edits Contributor

Motivation.

TL;DR: Introducing a fully SPMD-style LLMEngine execution pattern to improve offline inference throughput.

The RFC draft is initiated by @PeterSH6

vLLM in veRL(HybridFlow) (Fully SPMD)



× N

Driver and Worker are integrated in the same GPU process (Multi-Controller)

Support torchrun and SPMD-style offline inference #12071

Merged youkaichao merged 29 commits into vllm-project:main from youkaichao:torchrun_support on Jan 16, 2025

Conversation 28 Commits 29 Checks 10 Files changed 14

youkaichao commented on Jan 15, 2025 · edited by github-actions bot Member

Per the request from RLHF community #11400 , there's a strong need to co-locate vllm process and another process like training processes.

This PR enables the support, by allowing vllm launched in a `torchrun` -compatible way .

11

distributed_executor_backend="external_launcher"

vLLM changes for Verl

[Feature]: Avoid KV Cache and offload Model weights in RL workloads #11638

Closed



PeterSH6 opened on Dec 31, 2024 · edited by PeterSH6 Edits

🚀 The feature, motivation and pitch

Thanks for the awesome inference library! I'm writing to request two features that would be beneficial to RL post-training workloads.

In online PPO (or GRPO, online DPO), the policy model will perform auto-regressive generation (using vLLM or other inference engines) and fwd + bwd computation with training infrastructure. Therefore, in the training stage, we hope to free the KVCache and even offload the model parameter stored in the vLLM (as the model parallel strategies during generation and training could be different).

[Feature]: API for evicting all KV cache from GPU memory (or `sleep mode`) #10714

Closed #12695

HollowMan6 opened on Nov 27, 2024 · edited by HollowMan6 Edits Contributor

🚀 The feature, motivation and pitch

It will be great if we have an API to support evicting all KV cache from GPU memory.

By mentioning `sleep mode`, I mean, if there are some technical considerations, it's OK to make the vLLM unavailable for inference during this period, until we manually switch back to work by another separate API call. Ideally, the GPU memory usage during this period should be minimal, but the time for it to return back to normal should still be very fast.

[Core] Support fully transparent sleep mode #11743

Merged **youkaichao** merged 79 commits into `vllm-project:main` from `youkaichao:cumem` on Jan 22, 2025

Conversation 66 Commits 79 Checks 4 Files changed 14



youkaichao commented on Jan 5, 2025 • edited by github-actions bot Member

there is a strong need to put vllm in sleep mode (offload weight, discard kv cache), from rlhf community. see #10714 and #11638

I have implemented the core functionality in https://github.com/vllm-project/vllm_allocator_adaptor , and add the integration in this PR.

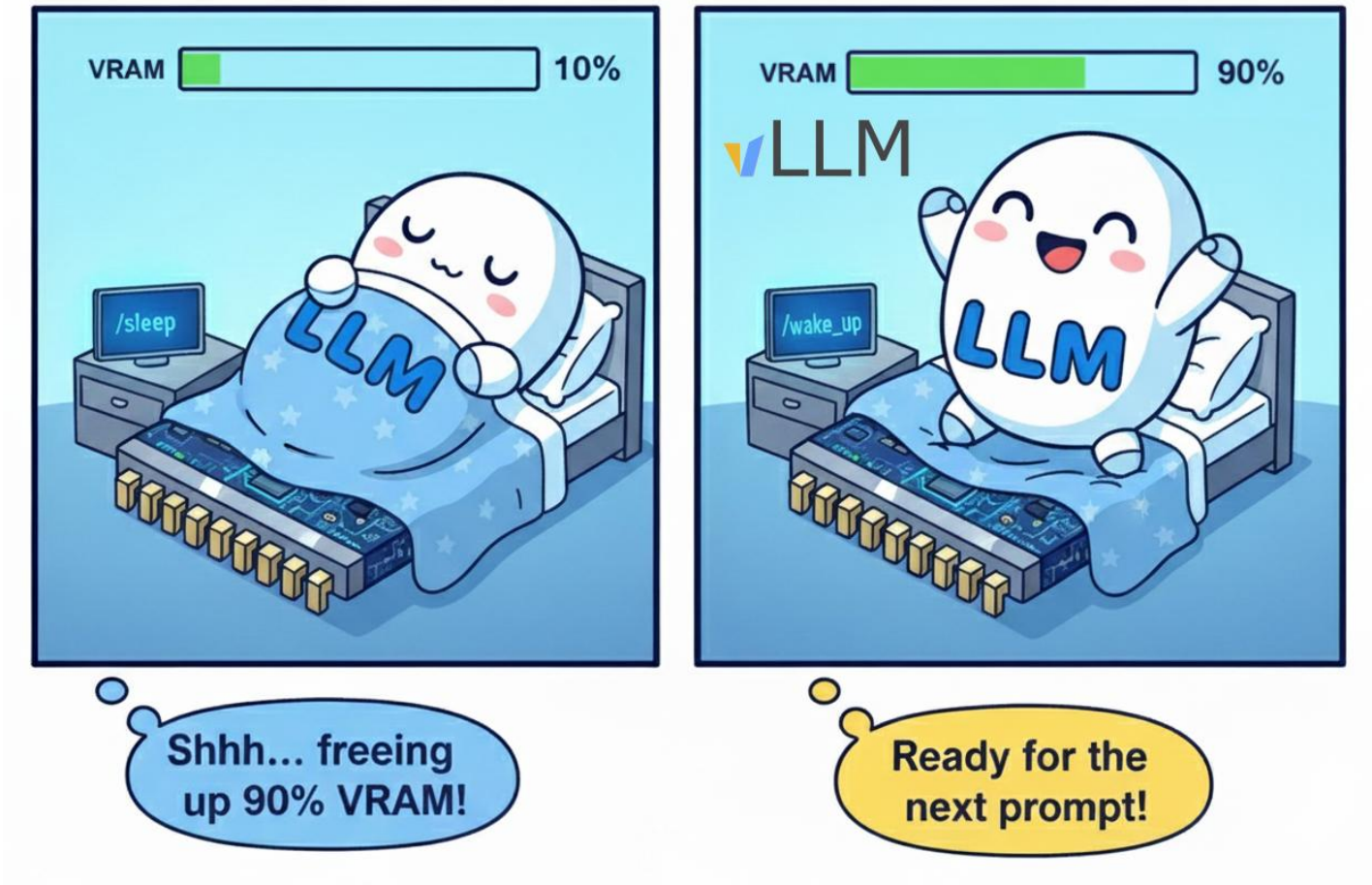
Currently, we support:

- offload model weights
- discard kv cache

NOTE: we do nothing for cudagraph memory right now.

Because the underlying `cumem` API is very low-level, this PR is also compatible with cudagraph.

The Impact beyond Verl



https://blog.vllm.ai/2025/10/26/sleep-mode.html

Blossom of RLHF frameworks







PRIME-RL



unsloth

OPENRLHF

Pipeline RL







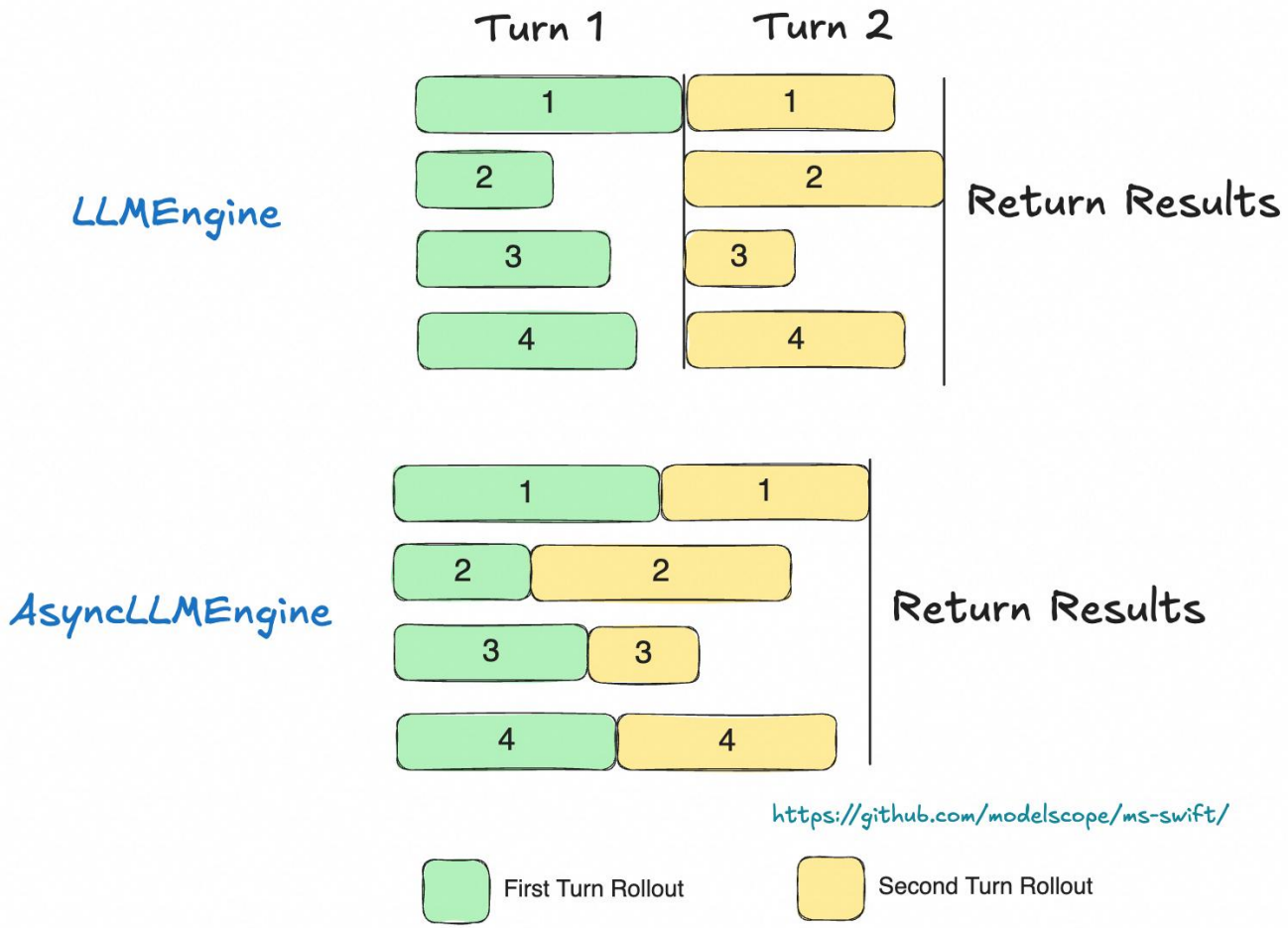


Nemo RL



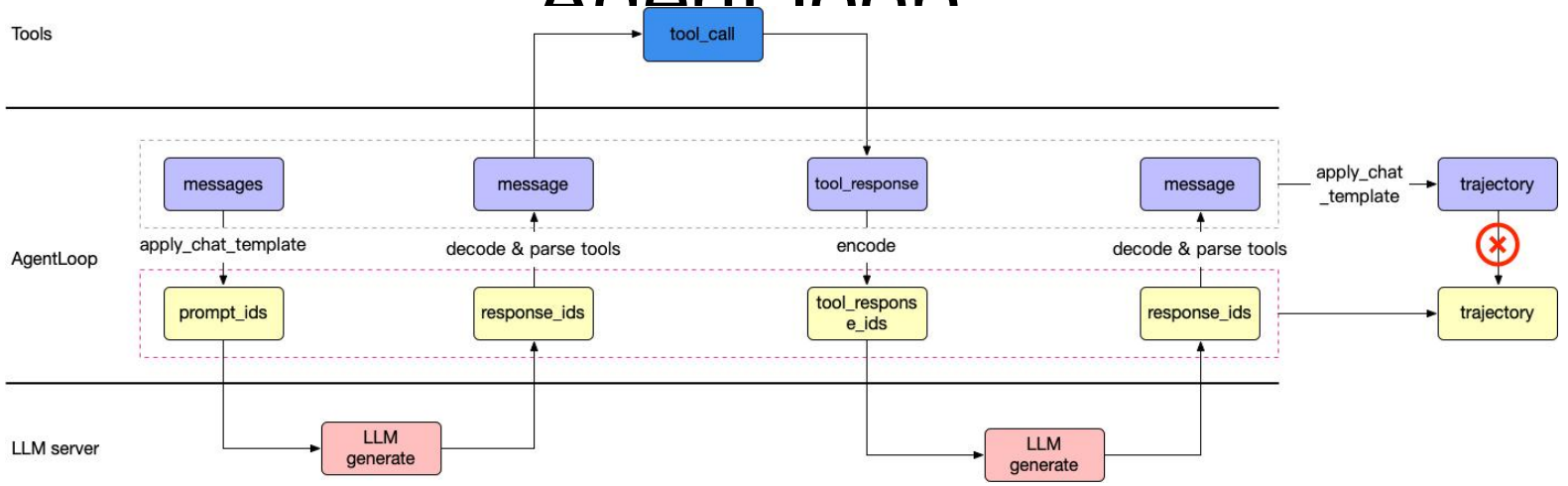
ROLL
like a Reinforcement Learning Algorithm Developer

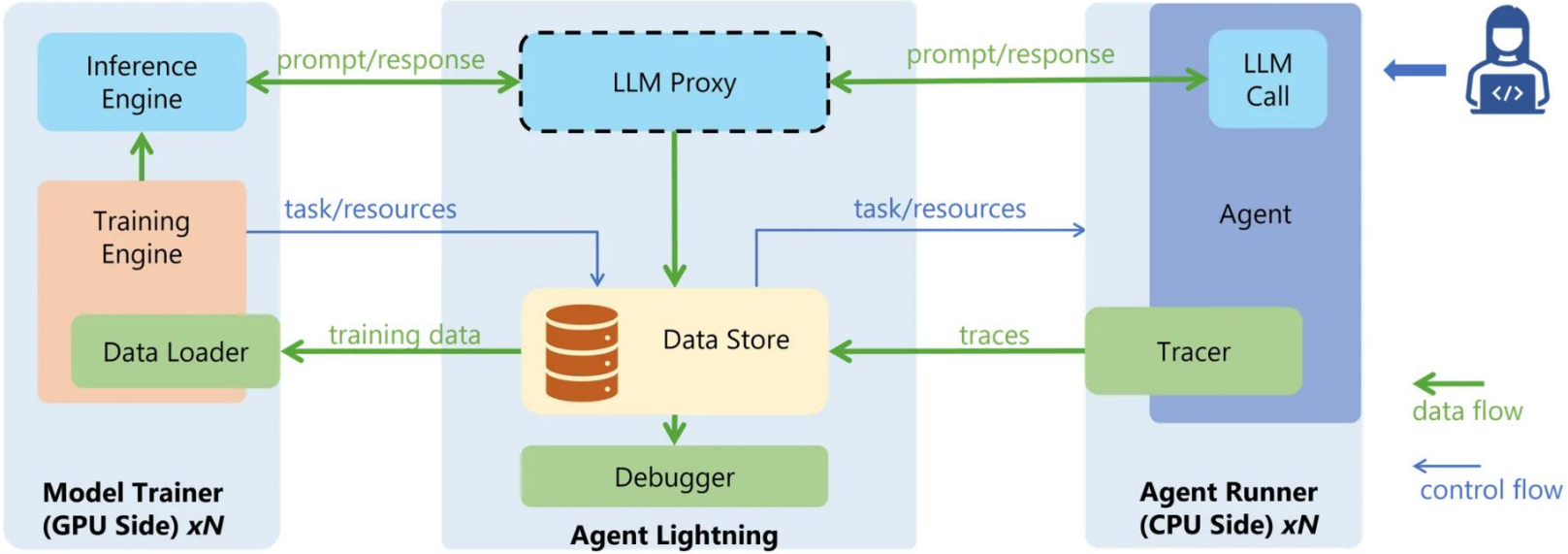
Advanced Usage of vLLM in RL: Multi-turn RL



https://swift.readthedocs.io/en/latest/Instruction/GRPO/DeveloperGuide/multi_turn.html

Advanced Usage of vLLM in RL: Token-in-token-out + Agent loop





https://verl.readthedocs.io/en/latest/advance/agent_loop.html

https://blog.vllm.ai/2025/10/22/agent-lightning.html

Async RL + Off-Policy Control + Partial Rollout

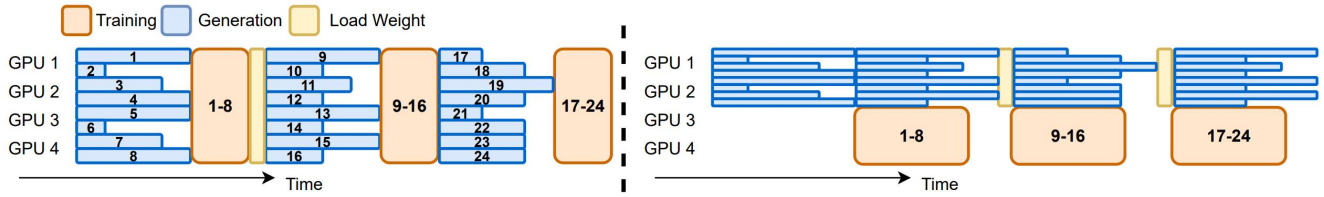


Figure 1: Execution timeline of a synchronous (left) and a one-step overlap (right) RL system showing underutilized inference devices.

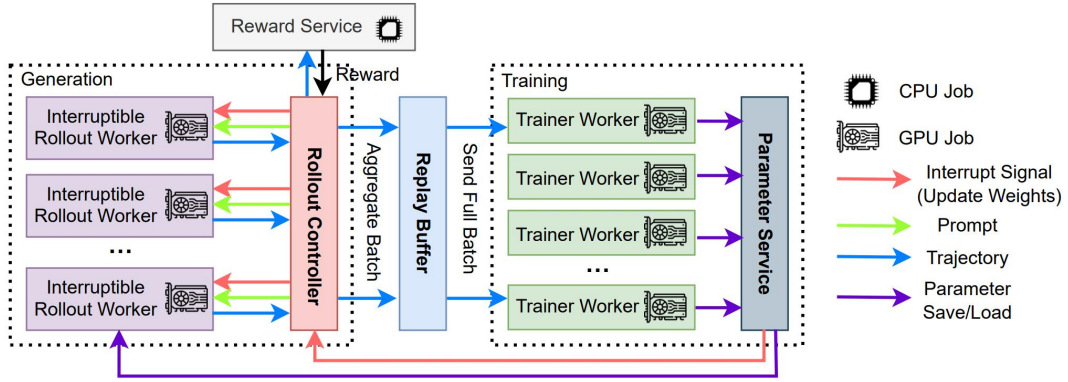
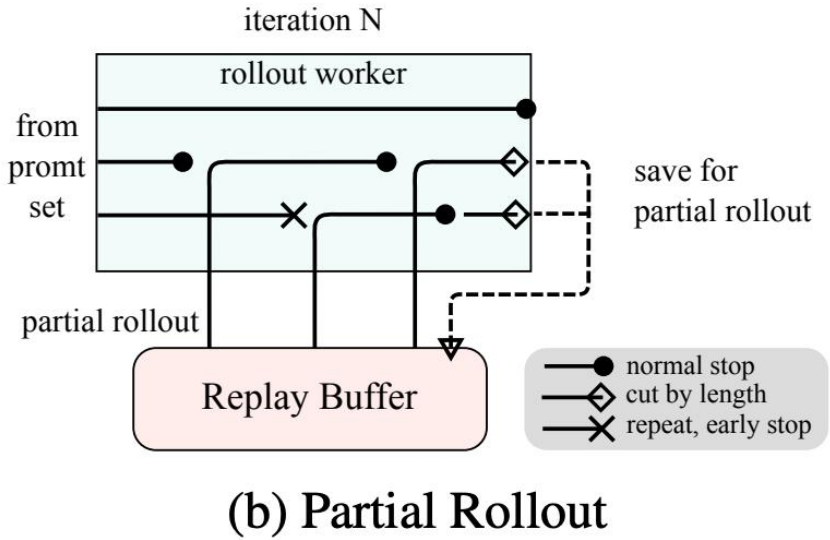


Figure 2: The AReaL architecture featuring asynchronous generation and training components.



(b) Partial Rollout

Wei Fu, etc. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning

Kimi Team: Kimi k1.5: Scaling Reinforcement Learning with LLMs

Crazy Ideas: In-flight Weight Update (with stale kvcache)

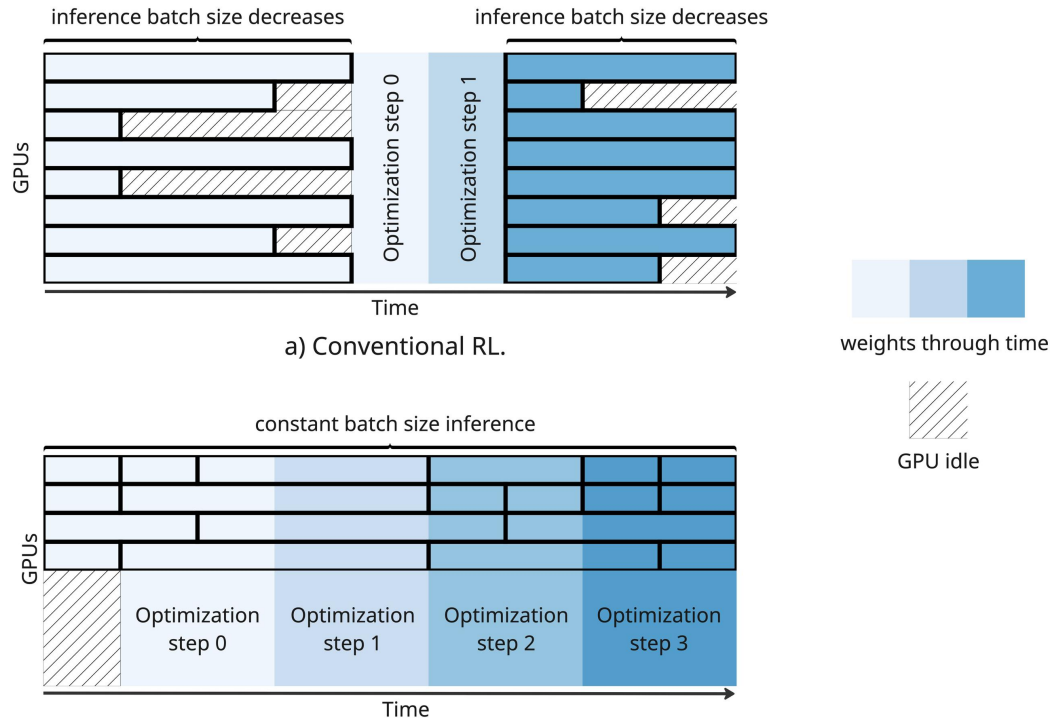


Figure 1. a) Conventional RL: Inference batch size decreases as shorter sequences complete and only longer sequences remain, reducing throughput, then the collected data becomes off-policy as weights are being update. b) Pipeline RL: Inference runs at full batch size maintaining high throughput, with weight updates after each optimizer step ensuring on-policy or near on-policy data collection.

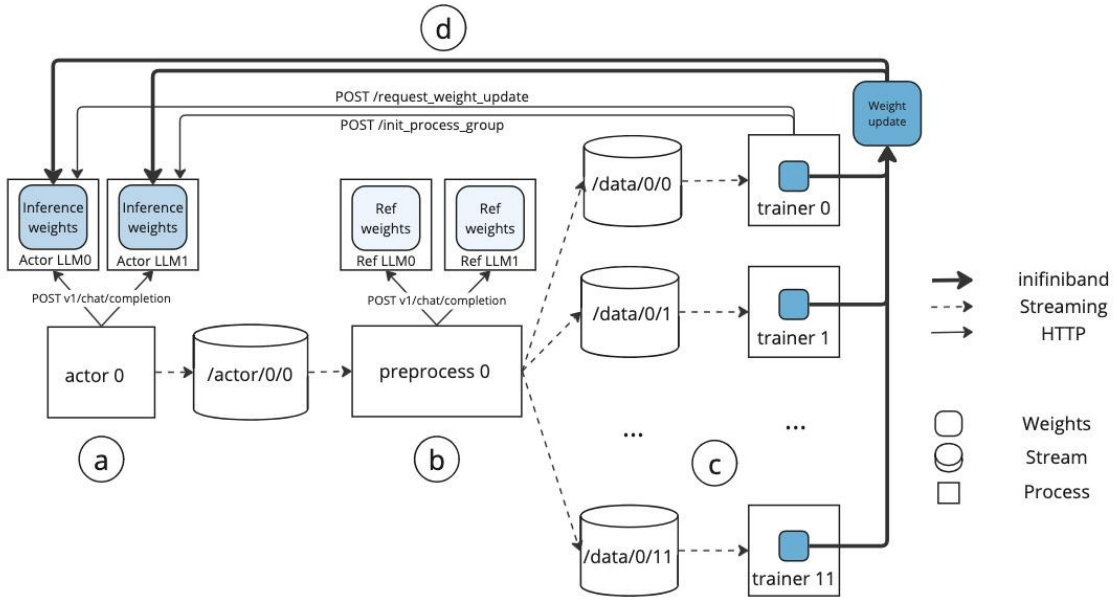


Figure 3. In a), the actor process sends HTTP requests to the actor LLM processes. The resulting rollouts are then streamed to the preprocessor. In b), the preprocessor annotates the rollouts with reference log probabilities which can be used by the trainer. In c), the trainers are continuously consuming the streamed data from the preprocessor and performing gradient steps. In d), the trainer 0 is sending HTTP requests to the Actor LLM processes 1) to initialize the process group (only done once at the beginning) and 2) to request a weight update. Finally, the weights are aggregated and transferred to update the inference weights.

https://huggingface.co/blog/ServiceNow/pipelinerl

Train/Inference Mismatch

Your Efficient RL Framework Secretly Brings You Off-Policy RL Training

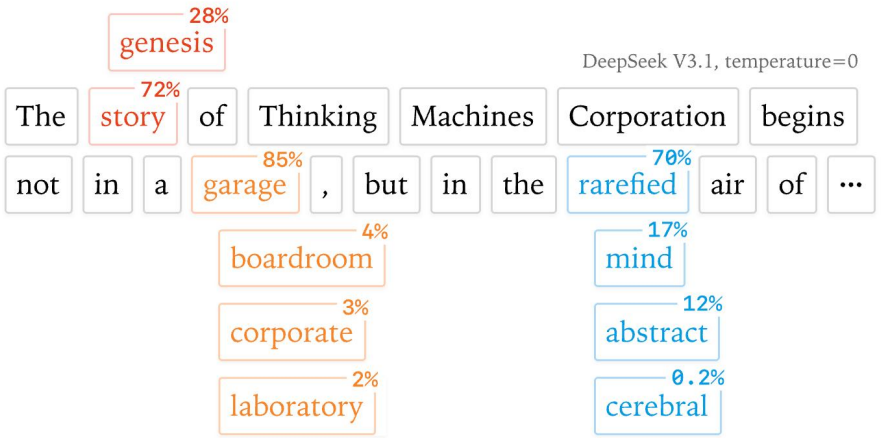
Feng Yao* Liyuan Liu* Dinghuai Zhang Chengyu Dong Jingbo Shang Jianfeng Gao

*: Equal Contributions (Work in Progress)

Defeating Nondeterminism in LLM Inference

Horace He in collaboration with others at Thinking Machines

Sep 10, 2025



When Speed Kills Stability: Demystifying RL Collapse from the Training-Inference Mismatch



Authors: Jiacai Liu* Yingru Li*† Yuqian Fu Jiawei Wang Qian Liu Zhuo Jiang†

Work done at ByteDance. First published at Sep 17, 2025.

```
VLLM_BATCH_INVARIANT=1 python simple_rl.py
```

logprobs_mode = "raw_logprobs" | "processed_logprob
"raw_logits" | "processed_logits"

Router Replay (WIP)

Keep Routing Mixture-of-Experts (MoE) models improve computational efficiency by activating only a subset of expert modules during inference. However, discrepancies between inference and training frameworks, compounded by policy updates, can result in inconsistent expert routing during inference and training even for identical inputs. Such inconsistency induces abrupt shifts in the active parameter subspace, which destabilizes optimization and exacerbates off-policy issues. To mitigate this, we preserve the expert routing paths used during sampling in the inference framework and enforce the same routing paths during training, ensuring that identical expert parameters are optimized. This Keep Routing operation was found crucial for RL training stability of MoE models, and has been adopted in our RL training pipeline since DeepSeek-V3-0324.

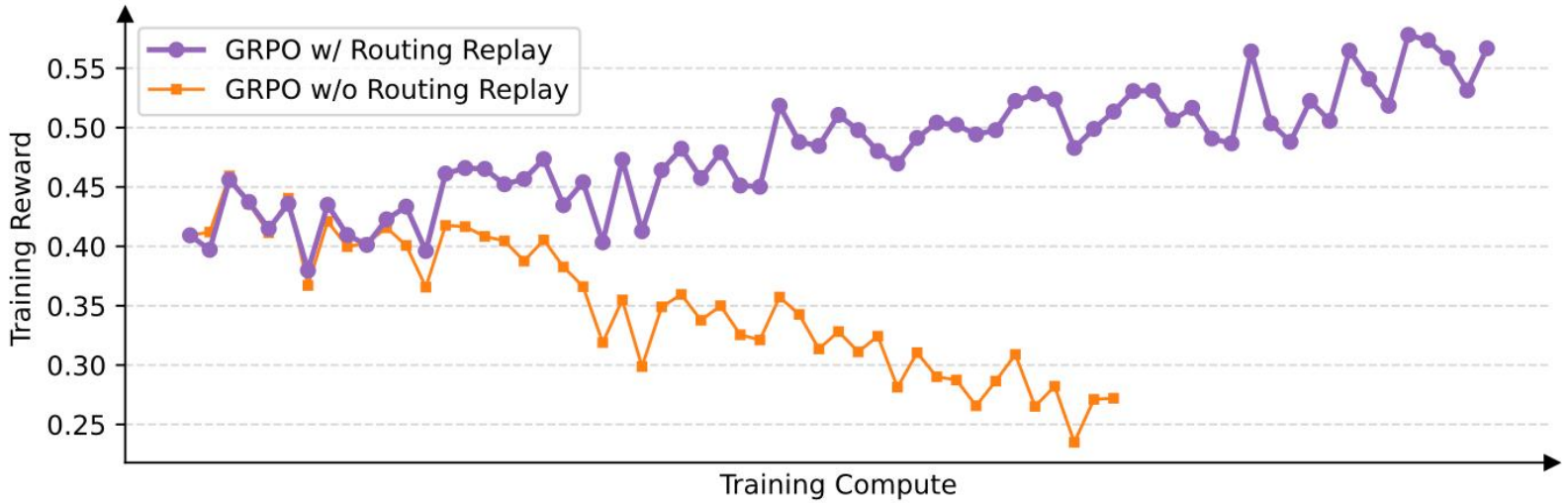


Figure 3: The Routing Replay strategy plays a critical role in the normal convergence of the GRPO training of MoE models.

Summarization of RL support in vLLM

- Driven by RL frameworks
- Provide functionalities for RL to control vLLM
- Use vLLM for rollout generation

- Future?
  - How to enable closer collaboration
  - How to explore advanced features in vLLM for acceleration
  - Mutli-modality RL

vLLM

Building the fastest and easiest-to-use open-source LLM inference & serving engine!

https://github.com/vllm-project/vllm

https://slack.vllm.ai

https://docs.vllm.ai

Ask AI

https://twitter.com/vllm_project

https://blog.vllm.ai

https://opencollective.com/vllm

https://www.zhihu.com/people/vllm-team

(vLLM 微信小助手) vllm_project

https://www.linkedin.com/company/vllm-project

collaboration@vllm.ai